CIS*6510 — Assignment 2: Cloud IAM Audit — Part A4 Writeup

Name: Clement Ampofo | Student ID: <your student id>

(a) The 2019 Capital One breach, mapped to IAM concepts

What happened. A web-application firewall running on an EC2 instance was misconfigured and vulnerable to *server-side request forgery* (SSRF). The attacker coerced that server into making an HTTP request to the EC2 *Instance Metadata Service* (IMDS, at `169.254.169.254`). IMDS returned the temporary credentials of the IAM role attached to the instance. That role was heavily over-privileged for S3, so the stolen credentials were used to list buckets and copy the data of ~100 million customers out of S3.

Mapping onto this assignment's model. The instance runs as an IAM role whose *trust policy* allows the service principal `ec2.amazonaws.com` to `sts:AssumeRole` — exactly the shape of `InfraAdminRole` in `account.json`. IMDS is the delivery channel that hands that role's short-lived credentials to any code that can make a request from the box; the SSRF bug gives the attacker that ability, so the attacker effectively *becomes* the instance role (an identity hand-off, like a one-step escalation edge). The role's *identity policy* then granted far more than the application needed — the broad `s3:*` / `Resource "*"` pattern that our Part A3 wildcard-budget rule forbids. The full chain is: **SSRF → IMDS credential theft → over-privileged instance role → bulk S3 read**.

The one policy change that most limits blast radius. Apply *least privilege to the instance role's S3 policy*: scope its `Resource` down from `"*"` (or whole-service `s3:*`) to only the specific bucket and prefix the application actually needs, e.g. `arn:aws:s3:::app-data/*`, and only the required actions (`s3:GetObject`). Then, even if the credentials are stolen via SSRF, they can touch one narrow dataset instead of every bucket — the damage is contained. (Hardening IMDS, e.g. requiring IMDSv2, blocks the SSRF step itself, but the question asks for the *policy* change, and resource-scoping the role is what shrinks the blast radius.)

(b) Why `iam:PassRole` is dangerous, and a control for it

Why it is dangerous. `iam:PassRole` lets a principal hand an existing IAM role to an AWS service when creating a resource that will *run as* that role (for example launching an EC2 instance with an instance profile, or creating a Lambda function with an execution role). The danger is *indirect privilege escalation*: a user who cannot perform action X directly, but who can pass a powerful role *and* start a service that assumes it, inherits everything that role can do. They create compute that executes with the passed role's permissions and drive it from there. This is precisely the `PassRole+RunInstances(EC2)` path (`bob → InfraAdminRole`) and the `PassRole+CreateFunction(Lambda)` path (`erin → ReportingLambdaRole`) found in Part A2: a low-privileged user reaches an admin-equivalent role without ever being granted admin directly.

A control that constrains it. Never grant `iam:PassRole` on `Resource "*"`. Scope the `PassRole` statement's `Resource` to a short, explicit list of specific, least-privileged role ARNs that the user legitimately needs to pass, and add a `Condition` on the `iam:PassedToService` key (e.g. `StringEquals iam:PassedToService = ec2.amazonaws.com`) so the role can only be handed to the intended service. In parallel, keep the passable roles themselves minimally privileged — which is how Part A3 closed these paths, by removing admin-equivalence from the target roles and the escalation-enabling permission pairings.